- **MySQL数据库——什么是数据库**
  - 数据库 (database) 是用来组织、存储和管理数据的仓库。
    - 当今世界是一个充满着数据的互联网世界，充斥着大量的数据。数据的来源有很多，比如出行记录、消费记录、浏览的网页、发送的消息等等。除了文本类型的数据，图像、音乐、声音都是数据。
    - 为了方便管理互联网世界中的数据，就有了数据库管理系统的概念(简称:数据库)。用户可以对数据库中的数据进行新增、查询、更新、删除等操作。
  - 增删改查
    - 新增 （create、add、insert）
    - 删除（delete）
    - 修改（update）
    - 查询（read　query）

- **MySQL数据库——常见的数据库及分类**
  - 市面上的数据库有很多种，最常见的数据库有如下几个:
    - MySQL 数据库(目前使用最广泛、流行度最高的的开源免费数据库;)
    - Oracle 数据库(收费)
    - SQL Server 数据库(收费)
    - Mongodb 数据库(Community + Enterprise)
  - 其中，MySQL、Oracle、SQL Server 属于传统型数据库(又叫做:关系型数据库 或 SQL 数据库)，这三者的 设计理念相同，用法比较类似。
  - 而 Mongodb 属于新型数据库(又叫做:非关系型数据库 或 NoSQL 数据库)，它在一定程度上弥补了传统型 数据库的缺陷。

- **MySQL数据库——MySQL简介**
  - MySQL是一个关系型数据库管理系统，由瑞典MySQL AB 公司开发，目前属于 Oracle 旗下产品。
  - 我们常说数据库，其实只是一个泛指。那么数据库的结构是怎样的呢?
    - 数据库服务器>数据库>数据表（真正存储数据的地方）

- 数据表的结构和excel一模一样。

id（只能填数字，不允许重复）	name（字符串、10、必填）	age	sex	tel
1	王宇	23	男	13200008888
2	王宇	23	男	13300008888
3	裴志博	25	男	18866669999
4	李淑茵	32	女	13200008888

- **安装MySQL及Navicat**
  - 安装MySQL服务软件
    - 安装phpStudy，或者wampserver。二选一。
    - 安装过程，略
  - 安装操作MySQL的图形化工具（Navicat）
    - 图形化的管理工具，有很多种
      - mysql-workbeach（英文版，没有中文版）
      - Navicat
      - phpmyadmin（需要php支持）
      - 其他，基本都不跨平台
    - 前面已经安装了MySQL软件。那么我们如何管理或者说使用它呢，对于我们来说，还需要安装一个管理MySQL的工具，我们选择就是 `Navicat`。
    - Navicat是一个收费软件，我们可以免费试用 14 天。
    - MySQL服务和图形化工具的关系

- **Navicat使用**
  - 必要条件
    - 必须启动MySQL服务。
      - 如果你使用phpstudy，打开phpstudy，启动MySQL。
      - 如果你使用wampserver，打开wampserver软件，MySQL就启动了
  - 连接到MySQL服务器
    - 打开 Navicat软件，点击连接 --> MySQL。
    - 填写如下参数：
      - 连接名：随便填。
      - 主机：localhost （不用改）
      - 端口：3306 （不用改）
      - 用户名：root （不用改）
      - 密码：自己的密码是什么，就填什么。（phpstudy默认密码root、wampserver默认密码空）
    - 填好连接参数，可以点左下角的 “测试连接”，如果成功了，点击右下角的“保存”即可。
    - 至此，Navicat侧边栏就有一个连接了。
    - 双击或者右键打开这个连接，就表示使用Navicat软件连接到MySQL数据库了，后面就可以管理数据库了。
  - 创建数据库
    - 在连接名称上，右键，选择 “新建数据库”
    - 只需填数据库名，选择utf8编码，然后确定
  - 创建数据表
    - 对于前端同学来说，创建数据表只需了解即可。
    - 比如创建一个学生信息表：

id（不允许重复）	name	age	sex	tel
1	王宇	23	男	13200008888
2	王宇	24	男	13300008888
3	裴志博	25	男	18866669999
...	...	...	...	...

    - 下面是关于表头的设计：

名（表头）	类型	长度	不是null	键	其他
id	int		√	🔑	√自动递增
username	varchar	20	√		
age	int				
sex	varchar	1			

        - id -- 自动递增 -- √
        - 最后保存，填表名 `student`

- **导入导出数据表**
    - 导出
        - 在数据表名字上，比如 `student` 上，右键 --> 转储SQL文件 --> 结构和数据，选择保存位置保存即可。
    - 导入
        - 在`数据库名`上面 --> 右键 --> 运行SQL文件 --> 选择SQL文件，运行即可完成导入。
        - 导入注意事项，表名不能重复，如果重复会发生覆盖。

- **SQL语句**
    - SQL(英文全称:Structured Query Language)是结构化查询语言，专门用来访问和处理数据库的编程语言。
    - SQL能做什么
        - 从数据库中查询数据
        - 向数据库中插入新的数据
        - 更新数据库中的数据
        - 从数据库删除数据
        - 可以创建新数据库
        - 可在数据库中创建新表
        - 可在数据库中创建存储过程、视图
        - etc...

- **查询数据**
    - 语法格式
        - SQL语句，`不区分`大小写。

```
-- 基本的查询语法
SELECT 字段1,字段2,... FROM 表名
-- 不区分大小写
```

```
select 字段,字段,.... from 表名

-- 查询所有的字段
SELECT * FROM 表名

-- 带条件的查询
SELECT * FROM 表名 [WHERE 条件] [ORDER BY 排序字段[, 排序字段]] LIMIT [开始位置,]长度
```

- 基本查询
  - 语法：select 字段名1, 字段名2,.... from 表名
  - 案例1: 查询所有学生的姓名和年龄

```
select username,age from student
```

  - 案例2: 查询全部学生的全部信息

```
select * from student
```

- 带条件的查询
  - 语法：select 字段 from 表名 where 条件
  - 可以使用条件来筛选查询出的结果

```
-- 查询id小于10的学生
-- select * from student where 条件
-- select * from student where id<10

-- 查询id小于20的女学生
-- select * from student where id<20 and sex='女'

-- 查询年龄大于等于20小于等于30的学生
-- select * from student where age>=20 and age<=30
```

- 对查询结果排序
  - 语法：select 字段 from 表名 order by 字段 规则 [,字段 规则 [,字段 规则 [.....]]]
  - 规则只有下面两种：
    - 升序 asc （默认值

        - 降序 desc
    - 可进行排序的字段通常是 整型 英文字符串型 日期型 (中文字符串也行,但一般不用)

```
-- 对查询结果进行排序
-- 查询所有的同学，并按年龄升序排序
-- select * from student order by age asc
-- select * from student order by age

-- 查询所有的同学，按年龄降序排序
-- select * from student order by age desc

-- 查询所有的同学，按年龄降序排序，如果年龄相同，按id降序排序
-- select * from student order by age desc, id desc

-- 如果SQL中既有条件、又有排序，必须先写条件
-- 查询所有的男同学，并按年龄升序排序
select * from student where sex='男' order by age asc
```

    - 注意：如果SQL语句中，有where和order by，where一定要放到order by之前。

- **添加数据**
    - 语法： insert into 表名 set 字段=值, 字段=值, ...

```
-- insert into 表名 set 字段=值, 字段=值, ....

insert into student set age=30, sex='男', username='李青'
```

- **修改数据**
    - 语法：update 表名 set 字段=值, 字段=值,..... where 修改条件（不指定修改条件会修改所有的数据）

```
-- 修改id为11的数据
update student set age=20, sex='女' where id=11

-- 没有指定条件，全部的数据都会修改
update student set age=25, sex='女'
```

- **删除数据**
  - 语法：delete from 表名 where 删除条件（不指定条件将删除所有数据）

```
-- 删除一条数据
delete from student where id=11

-- 删除满足条件的数据
delete from student where id>6

-- 没有指定条件，删除全部数据
delete from student
```

- **SQL小结**
  - SQL相当于是数据库中使用的编程语言。
  - 可以通过SQL完成各项工作，比如查询数据，新增数据，删除数据，修改数据.....
  - 常用的增删改查语句：
    - 查询 （`select 字段, 字段,... from 表名 [where 条件] [order by 字段 排序规则]`）
      - select * from student where id>5 order by age desc
    - 新增（`insert into 表名 set 字段=值, 字段=值, ....`）
      - insert into student set username='张三', age=20, sex='男'
    - 修改（`update 表名 set 字段=值, 字段=值, .... [where 条件]`）
      - update student set username='李四', sex='女' where id=4
    - 删除（`delete from 表名 [where 条件]`）
      - delete from student where id=3

- **Node中的mysql模块**
  - mysql模块的作用
    - 数据在数据库中保存着呢?
    - 但最终，用户应该在浏览器界面上看到数据。
    - 这就需要使用 JS 代码，将数据库中的数据查询出来。
    - mysql模块是一个第三方模块，专门用来操作MySQL数据库。 可以执行增删改查操作。
  - 安装mysql模块
    - 初次安装第三方模块，只需要按照如下方式安装即可。后续会有详细的介绍。

```
# 注意，安装mysql的文件夹，不能用中文，不能叫mysql（不能和模块同名）
```

```

# 最好先执行下面这条命令，会帮你提高下载速度
npm config set registry https://registry.npm.taobao.org

# 初始化
npm init -y

# 执行下面的命令，下载安装mysql
npm i mysql
```

- 使用步骤
  - 在Node中使用MySQL模块一共需要5个步骤：
    - 加载 MySQL 模块
    - 创建 MySQL 连接对象
    - 连接 MySQL 服务器
    - 执行SQL语句
    - 关闭链接

```
// 1. 加载mysql模块
const mysql = require('mysql');
// 2. 创建连接对象（设置连接参数）
const conn = mysql.createConnection({
    // 属性：值
    host: 'localhost',
    user: 'root',
    password: '密码',
    database: '数据库名'
});

// 3. 连接到MySQL服务器
conn.connect();

// 4. 完成查询（增删改查）
conn.query(SQL语句, (err, result) => {
    err: 错误信息
    result: 查询结果
});

// 5. 关闭连接，释放资源
```

```
conn.end();
```

- 增删改查小结
  - 查询语句
    - result -- 数组
    - 数组的每个单元，就是查询到的一行数据
  - 添加语句
    - result -- 对象
    - result.affectedRows -- 受影响的行数
    - result.insertId -- 新增数据id
  - 修改语句
    - result -- 对象
    - result.affectedRows -- 受影响的行数（满足条件的）
    - result.changedRows -- 被改变的行数（真正发生变化的行数）
  - 删除语句
    - result -- 对象
    - result.affectedRows -- 受影响的行数
- 封装MySQL
  - 封装mysql，然后导出

```
module.exports = function (sql, callback) {
    const mysql = require('mysql');
    const conn = mysql.createConnection({
        host: 'localhost',
        user: 'root',
        password: '12345678',
        database: 'hahaha'
    });
    conn.connect();
    // 完成增删改查
    conn.query(sql, callback);
    conn.end();
}
```

  - 创建一个测试的文件，试试封装的函数

```
// 加载自定义模块
```

```
const db = require('./db');

// 调用函数
db('select * from student where id<5', (err, result) => {
  if (err) throw err;
  console.log(result);
});
```